```
"================================================================================
" CONFIGURAÇÃO NEOVIM FINAL - CERBERO (M3 MAX) - VERSÃO CORRIGIDA
"================================================================================

let g:loaded_netrw = 1
let g:loaded_netrwPlugin = 1

set mouse=a
set number relativenumber
set termguicolors
set laststatus=2
syntax on
filetype plugin indent on
set clipboard=unnamedplus
let mapleader = ' '

" --- WRAP VISUAL GERAL ---
set wrap
set linebreak
set breakindent
set breakindentopt=min:20

" --- GERENCIADOR DE PLUGINS ---
call plug#begin(expand('~/.local/share/nvim/site/plugged'))
Plug 'nvim-treesitter/nvim-treesitter', {'do': ':TSUpdate'}
Plug 'digitaltoad/vim-pug'
Plug 'lukas-reineke/indent-blankline.nvim'
Plug 'nvim-lua/plenary.nvim'
Plug 'nvim-telescope/telescope.nvim', { 'tag': '0.1.8' }
Plug 'nvim-tree/nvim-web-devicons'
Plug 'nvim-tree/nvim-tree.lua'
Plug 'neovim/nvim-lspconfig'
Plug 'williamboman/mason.nvim'
Plug 'williamboman/mason-lspconfig.nvim'
```

```
Plug 'hrsh7th/nvim-cmp'
Plug 'hrsh7th/cmp-nvim-lsp'
Plug 'hrsh7th/cmp-buffer'
Plug 'L3MON4D3/LuaSnip'
call plug#end()

" --- CORES E INTERFACE ---
silent! colorscheme blue
hi Normal guibg=NONE
set colorcolumn=7,8,12,72
highlight ColorColumn guibg=#5f0000 ctermbg=52
highlight IblIndent guifg=#51afef gui=nocombine
highlight IblScope guifg=#00ffff gui=nocombine
set guicursor=n-v-c-i:block-Cursor
highlight Cursor guibg=#ffff00 guifg=black

lua << LUA
-- 1. MASON & LSP (Gerenciador de Servidores)
local status_mason, mason = pcall(require, "mason")
local status_mason_lsp, mason_lsp = pcall(require, "mason-lspconfig")
local status_lspconfig, lspconfig = pcall(require, "lspconfig")

if status_mason and status_mason_lsp and status_lspconfig then
    mason.setup()
    mason_lsp.setup({
        ensure_installed = { "cobol_ls" },
        handlers = {
            function(server_name)
                lspconfig[server_name].setup {}
            end
        }
    })
end

```

```
-- 2. AUTOCOMPLETE (CMP)
local status_cmp, cmp = pcall(require, "cmp")
local status_luasnip, luasnip = pcall(require, "luasnip")

if status_cmp and status_luasnip then
    cmp.setup({
        snippet = { expand = function(args) luasnip.lsp_expand(args.body) end },
        mapping = cmp.mapping.preset.insert({
            ['<C-Space>'] = cmp.mapping.complete(),
            ['<CR>'] = cmp.mapping.confirm({ select = true }),
            ['<Tab>'] = cmp.mapping.select_next_item(),
            ['<S-Tab>'] = cmp.mapping.select_prev_item(),
        }),
        sources = cmp.config.sources({ { name = 'nvim_lsp' }, { name = 'buffer' } })
    })
end

-- 3. INTERFACE (Nvim-Tree, Treesitter, Indent-Blankline)
local status_tree, nvim_tree = pcall(require, "nvim-tree")
if status_tree then
    nvim_tree.setup({ sync_root_with_cwd = true })
end

local status_ts, ts = pcall(require, "nvim-treesitter.configs")
if status_ts then
    ts.setup({
        ensure_installed = { "cobol", "python", "lua", "vim", "vimdoc" },
        highlight = { enable = true },
        indent = { enable = true } -- Ativa indentação inteligente do treesitter
    })
end
```

```
100
101 -- 1. CONFIGURAÇÃO DOS PONTINHOS E LINHA AZUL
102 local status_ibl, ibl = pcall(require, "ibl")
103 if status_ibl then
104     -- Define os caracteres invisíveis (os pontinhos)
105     vim.opt.list = true
106     vim.opt.listchars:append({ space = "·" }) -- Define o
    ponto para espaços
107
108     ibl.setup {
109         indent = {
110             char = "│",          -- A linha vertical
      descendente
111             highlight = { "IblIndent" }, -- Usa a cor azul
      que definimos
112         },
113         whitespace = {
114             highlight = { "IblIndent" }, -- Aplica azul
      também nos pontinhos
115             remove_blankline_trail = false,
116         },
117         scope = { enabled = false }, -- Desativa o escopo
     para focar na indentação pura
118     }
119 end
120
121 local status_tel, telescope = pcall(require, "telescope")
122 if status_tel then
123     telescope.setup({
124         defaults = {
125             preview = {
126                 treesitter = false, -- Desativa o
       Treesitter apenas na janela de visualização
127             },
128         },
129     })
130 end
```

```
LUA

" --- ATALHOS GERAIS ---
nnoremap <Tab> <C-w>w
nnoremap <S-Tab> <C-w>W
nnoremap <C-b> :vsp<cr>
nnoremap <C-d> :belowright split<cr>
nnoremap <C-s> :w!<cr>
nnoremap <C-q> :q<cr>
nnoremap <C-n> :NvimTreeToggle<CR>
nnoremap <C-p> :lua
require('telescope.builtin').find_files({ cwd =
vim.fn.expand('~/COBOL/cob') })<CR>

" --- AMBIENTE PYTHON ---
nnoremap <C-r> :update <bar> botright split <bar> term
python3 %<CR>
nnoremap <F5> :update <bar> botright split <bar> term
python3 %<CR>
tnoremap <Esc> <C-\><C-n>
autocmd TermOpen * startinsert

" --- AMBIENTE COBOL (COMPILAÇÃO E WRAP FÍSICO) ---
let g:cobol_autoupper = 0
let g:cobol_fold = 0

function! RodarCobol()
    write
    let l:arquivo_cob = expand('%:p')
    let l:script_sh = expand('~/COBOL/setup_cobol.sh')
    let l:comando = "botright split | term " .
  l:script_sh . " '" . l:arquivo_cob . "'"
    execute l:comando
endfunction

augroup cobol_logic
    autocmd!
```

```
163     " Unificação: Aplica margem 72, wrap físico (t) e
    autoindent em um só bloco
164     autocmd FileType cobol setlocal textwidth=72
    formatoptions+=tc autoindent
165     autocmd FileType cobol setlocal wrap linebreak
    colorcolumn=7,8,12,72
166     " Atalho F10
167     autocmd FileType cobol nnoremap <buffer> <F10> :call
    RodarCobol()<CR>
168 augroup END
169
170 " --- TEMPLATE COBOL ---
171 function! CreateCobolTemplate()
172     if line('$') == 1 && getline(1) == ''
173         let l:prog = toupper(expand("%:t:r"))
174         call append(0, ["       IDENTIFICATION DIVISION.",
      "       PROGRAM-ID. " . l:prog . ".", "       AUTHOR.
      CERBERO.", "       *", "       ENVIRONMENT DIVISION.",
      "       *", "       DATA DIVISION.", "       WORKING-
      STORAGE SECTION.", "       *", "       PROCEDURE
      DIVISION.", "       MAIN-PROCEDURE.", "
      DISPLAY 'OLA NEOVIM'.", "           STOP RUN.", "
      END PROGRAM " . l:prog . "."])
175         call cursor(12, 11)
176     endif
177 endfunction
178 autocmd BufNewFile *.cob,*.cbl call CreateCobolTemplate()
179
180 " --- EVOLUÇÃO MÉDICA ---
181 function! GerarEvolucaoMedica()
182     let l:paciente = input('Nome do Paciente: ')
183     if l:paciente == '' | return | endif
184     let l:data = input('Data (DD/MM/AAAA): ')
185     if l:data == '' | return | endif
186
187     let l:paciente_und = substitute(l:paciente, ' ', '_',
    'g')
```

```
188     let l:data_slug = substitute(l:data, '/', '-', 'g')
189     let l:script = expand('~/EXAMESPDF/gerar_evolucao.py')
190     let l:arquivo_txt = expand('~/EXAMESPDF/') .
    l:paciente_und . '/evolucao_' . l:paciente_und . '_' .
    l:data_slug . '.txt'
191
192     let l:cmd = 'python3 ' . shellescape(l:script) . ' ' .
    shellescape(l:paciente) . ' ' . shellescape(l:data)
193     call system(l:cmd)
194
195     if filereadable(l:arquivo_txt)
196         call append(line('.'), readfile(l:arquivo_txt))
197         call delete(l:arquivo_txt)
198         echo "\nEvolução inserida!"
199     else
200         echo "\nErro ao localizar arquivo."
201     endif
202 endfunction
203 nnoremap <leader>ev :call GerarEvolucaoMedica()<CR>
204
```